

PROCEDIMIENTO DE IMPLEMENTACIÓN DE SOFTWARE

RENTAMOVIL

**Thiago Fabian Rojas Guevara — David Santiago Erez Rodríguez — Nicole Dayana Ramírez
Vargas**

ANÁLISIS Y DESARROLLO DE SOFTWARE 3145556

SERVICIO NACIONAL DE APRENDIZAJE

2026

1. OBJETIVO DEL DOCUMENTO

Establecer de forma clara y verificable los pasos técnicos y operativos seguidos para desplegar, configurar y poner en funcionamiento el aplicativo RentaMóvil (Web y Móvil) de forma accesible públicamente, documentando exactamente lo que se ejecutó y se comprobó funcionando — no un plan aspiracional.

2. ALCANCE

Este procedimiento abarca desde la preparación de la infraestructura (Docker, red compartida, proxy inverso) hasta la publicación pública del front web y la API, y la distribución de un instalable Android de la app móvil.

2.1 Decisión de alcance: servidor propio + túnel, no nube ni tiendas

Por decisión del equipo, el despliegue usa una máquina Linux propia como servidor (no un proveedor cloud) con Docker + nginx como proxy inverso + ngrok para la URL pública. Esto evita depender de un host cloud con soporte para SQL Server (el motor real del proyecto, no incluido gratis en la mayoría de PaaS) y es coherente con el alcance académico del proyecto. La app móvil se distribuye como APK de instalación directa (EAS Build, perfil interno), sin pasar por Google Play / App Store — evita costos de cuenta de desarrollador y tiempos de revisión externa (riesgo R-004 en `rtm-docs/15-project-control/risks.md`).

2.2 Arquitectura del sistema desplegado

- **Frontend Web:** React + Vite, servido en producción ("vite preview") detrás de nginx.
- **Frontend Móvil:** Expo + React Native, compilado a un `.apk` de Android con EAS Build (perfil "preview", distribución interna).
- **Backend:** 4 microservicios Spring Boot (`rtm-iam`, `rtm-fleet-maintenance`, `rtm-booking-reservation`, `rtm-payment-billing`) detrás de un único API Gateway (`rtm-api-gateway`).
- **Base de datos:** una sola instancia SQL Server compartida, con un schema por servicio (ADR-009).
- **Publicación:** nginx como proxy inverso único + ngrok para la URL pública con dominio fijo reservado.

3. FASES DE LA IMPLEMENTACIÓN (EJECUTADAS Y VERIFICADAS)

Fase 1 — Preparación del entorno e infraestructura

Paso 1.1: Red Docker compartida

Se creó una única red Docker externa, usada por los 11 repositorios para encontrarse entre sí por nombre de servicio:

```
docker network create rentamovil
```

Paso 1.2: Cuenta y dominio de ngrok

Se creó una cuenta gratuita en ngrok y se reservó un dominio fijo (Dashboard → Domains → "+ New Domain"), para que la URL pública no cambie entre reinicios del túnel.

Fase 2 — Despliegue de bases de datos y servicios backend

Se levantó cada repositorio en el orden de dependencia real (foreign keys y schemas compartidos), verificado con `/health` del gateway tras cada paso:

#	Repositorio	Acción
1	rtm-iam-db	<code>docker compose up -d</code> — levanta el SQL Server compartido, crea la base rentamovil y migra el schema iam
2	rtm-fleet-maintenance-db	<code>docker compose up -d</code> — migra el schema fleet (depende de shared.audit, creado por iam-db)
3	rtm-booking-db	<code>docker compose up -d</code> — migra el schema booking (referencia fleet.vehicle / fleet.branch)
4	rtm-payment-billing-db	<code>docker compose up -d</code> — migra el schema payment (referencia booking.reservation)
5	rtm-iam	<code>docker compose up -d</code> — servicio de identidad y acceso
6	rtm-fleet-maintenance	<code>docker compose up -d</code> — servicio de flota y mantenimiento
7	rtm-booking-reservation	<code>docker compose up -d</code> — servicio de reservas
8	rtm-payment-billing	<code>docker compose up -d</code> — servicio de pagos
9	rtm-api-gateway	<code>docker compose up -d</code> — enruta a los 4 servicios anteriores, puerto público 8080
10	RentaMovil/web/Front-end	<code>docker compose up -d --build</code> — build de producción del front, puerto 4173
11	rtm-deploy	<code>docker compose up -d</code> — nginx (proxy inverso, puerto 80) + ngrok (túnel público con dominio fijo)

Verificación ejecutada tras el paso 9:

```
curl http://localhost:8080/health
→ {"status":"degraded","upstreams":{"iam":"ok","fleet-maintenance":"ok",
  "booking-reservation":"ok","payment-billing":"ok","telemetry-gps":"down"}}
```

"degraded" es el estado correcto y esperado: telemetry-gps es un servicio que rtm-docs documenta como objetivo futuro y que aún no existe como repositorio (ver 01-MAPA-GLOBAL-DEL-PROYECTO.md de las notas de estudio); los 4 servicios reales del alcance actual responden "ok".

Fase 3 — Publicación del front web y la API detrás de un único dominio

Paso 3.1: Enrutamiento de nginx

El front web ya resuelve sus llamadas a la API contra `/api` en su propio origen por defecto (mismo patrón que el proxy de Vite en desarrollo), así que nginx solo reproduce ese esquema de rutas sin tener que tocar código del front ni de la API:

Ruta pública	Destino (contenedor)	Para qué
<code>/</code>	web (:4173)	Front web — build de producción servido con "vite preview"
<code>/api/*</code>	api-gateway (:8080), sin el prefijo <code>/api</code>	Todo lo que ya tiene backend real: auth, usuarios, vehículos, sucursales, reservas, pagos, cuentas bancarias
<code>/mock/*</code>	mock-api (:3100), sin el prefijo <code>/mock</code>	Lo que aún no tiene servicio real (ver <code>REAL_BACKEND_PREFIXES</code> en <code>httpClient.js</code>)

Paso 3.2: Verificación de conectividad (ejecutada)

```
curl https://commerce-playmate-carving.ngrok-free.dev/  
→ HTTP 200 (index.html real del front)  
  
curl https://commerce-playmate-carving.ngrok-free.dev/api/health  
→ idéntico al curl directo a :8080/health
```

Incidente real durante esta fase (resuelto)

Bloqueo por "allowedHosts" de Vite:

Al abrir la URL pública por primera vez, "vite preview" devolvía 403 "Blocked request... not allowed" porque por defecto solo acepta peticiones con Host de localhost. Se corrigió agregando `allowedHosts: ['.ngrok-free.dev', '.ngrok.app', '.ngrok.io']` a la sección preview de `vite.config.js`, y reconstruyendo el contenedor web.

Fase 4 — Compilación y distribución de la app móvil (APK)

Paso 4.1: Configuración del proyecto EAS

```
npx eas-cli login  
npx eas-cli build:configure # crea eas.json, vincula el proyecto a la cuenta de Expo
```

Se ajustó el perfil "preview" de `eas.json` para generar un `.apk` instalable directo (por defecto Android compila `.aab`, que solo sirve para subir a Play Store):

```
"preview": { "distribution": "internal", "android": { "buildType": "apk" } }
```

Paso 4.2: Variable de entorno con la URL pública

La URL de la API queda fija dentro del binario compilado — se configuró antes de compilar:

```
EXPO_PUBLIC_API_URL=https://commerce-playmate-carving.ngrok-free.dev/api
```

Paso 4.3: Compilación

```
npx eas-cli build --platform android --profile preview
```

Incidente real durante esta fase (resuelto)

Fallo de Gradle por un asset corrupto:

El primer build falló en la fase "Run gradlew" con "AAPT: error: file failed to compile" sobre `assets/images/login.png`. La causa: el archivo era en realidad una imagen WebP con extensión `.png` (formato real distinto al declarado por el nombre), y el compilador de recursos de Android (aapt2) lo rechaza. Se corrigió convirtiendo el archivo a un PNG real (mismo contenido visual, formato correcto) y se relanzó el build, que completó exitosamente.

Paso 4.4: Instalación y verificación

El `.apk` se descargó desde el link que entrega EAS Build al finalizar, se instaló en un dispositivo Android (habilitando "orígenes desconocidos"), y se verificó el flujo real de la aplicación contra el backend público — funcionando correctamente.

Evidencia:

APK instalado y aplicación verificada funcionando contra el backend público (commerce-playmate-carving.ngrok-free.dev) — confirmado el 9 de octubre de 2026.

Fase 5 — Pruebas de humo en producción (realizadas)

- Health check del gateway a través de nginx: 200, 4 servicios reales "ok".
- Carga del front web por la URL pública: 200, contenido real servido.
- Flujo completo en la app móvil instalada (login, navegación, consumo de la API real) contra el backend público: verificado funcionando por el equipo.

4. PLAN DE MITIGACIÓN DE RIESGOS Y CONTINGENCIAS

Los riesgos de este despliegue están formalmente registrados en `enrtm-docs/15-project-control/risks.md`. Los más relevantes para esta implementación:

- **R-003** — SQL Server es una instancia única (sin alta disponibilidad): si se cae, los 4 servicios dejan de responder. Mitigación aplicada: procedimiento de recuperación documentado (`docker compose down + up`, no `restart`, en `rtm-iam-db`) — ya se ejecutó una vez durante esta misma implementación.
- **URL pública no fija (ngrok, plan gratuito)** — mitigado reservando un dominio fijo en la cuenta de ngrok y pasándolo explícito al contenedor (`--url=...` en `rtm-deploy/docker-compose.yml`), así la URL no cambia entre reinicios.
- **R-004** — Rechazo en tienda móvil: evitado por diseño — no se publica en tiendas en este alcance, se distribuye como APK firmado internamente vía EAS Build.
- **APK con URL de backend fija** — si el dominio público cambia, hay que recompilar y redistribuir el `.apk` — documentado como limitación conocida, no como bug.

5. CORRELACIÓN

- `rtm-deploy/README.md` (pasos de arranque detallados)
- `rtm-docs/10-devops/local-setup.md` (orden completo de los 10 repos)
- `rtm-docs/10-devops/environments.md` (estado real de entornos)
- `rtm-docs/15-project-control/risks.md` (R-003, R-004)